Interação Humano Computador & UX

Introdução

Neste material, vamos estudar sobre

- Interação Humano Computador
- Experiência do Usuário

Além de conhecer os conceitos na teoria, vamos desenvolver pequenas aplicações que os ilustram.

A origem

Donald Norman e **Jakob Nielsen** são dois dos principais profissionais responsáveis pelas definições e desenvolvimento da área.

Vejamos algo sobre eles. Visite os links a seguir para se familiarizar com sua carreira e feitos.

https://en.wikipedia.org/wiki/Don_Norman

https://en.wikipedia.org/wiki/Jakob_Nielsen_(usability_consultant)

É interessante observar que ambos foram responsáveis pela criação do **Nielsen Norman Group (NN/G)**. Veja.

https://en.wikipedia.org/wiki/Nielsen_Norman_Group

Definições

IHC

A **IHC** pode ser definida de **diferentes formas**. Veja algumas.

Definição 1
IHC é a disciplina que investiga o diálogo entre seres humanos e tecnologia. Ela busca entender como o design de interfaces pode criar experiências que não apenas funcionam, mas que se sintam naturais, intuitivas e emocionalmente conectadas ao contexto de quem as utiliza.

Definição 2
IHC é um campo multidisciplinar que combina computação, ciências comportamentais e design para planejar e implementar sistemas que respeitem as limitações e capacidades humanas. O objetivo é garantir que a tecnologia seja uma extensão das habilidades do usuário, eliminando barreiras de aprendizado.

Definição 3
IHC é a ciência de projetar tecnologia para pessoas. É o estudo de como transformar linhas de código e hardware em ferramentas que façam sentido para a mente humana, priorizando a usabilidade e a acessibilidade.

UX

UX vem de **User Experience** ou **Experiência do Usuário**. Ela também pode ser definida de **diferentes formas**.

Definição 1
UX é a soma de todas as emoções, crenças e reações que uma pessoa tem ao interagir com uma empresa, seus serviços ou seus produtos. Não se trata apenas de 'funcionar', mas de como a experiência ressoa ou ecoa na memória do usuário: foi frustrante, foi invisível ou foi prazeroso?

Definição 2
UX é o design do caminho completo. É a disciplina que garante que, desde o momento em que o usuário descobre uma necessidade até a conclusão de uma tarefa, não existam atritos. É sobre antecipar as necessidades do usuário e remover obstáculos antes mesmo que ele os perceba.

Definição 3
UX é o campo que estuda a relação entre as metas do usuário e a entrega da tecnologia. É o equilíbrio entre a utilidade (o que o sistema faz), a facilidade de uso (como ele faz) e o desejo (por que o usuário escolhe esse sistema em vez de outro).

Qual a diferença entre elas?

Observe que são definições relacionadas entre si. Uma tem tudo a ver com a outra. Uma pergunta natural é "**qual a diferença entre elas**?". Observe.

Interação Humano Computador VS User Experience
IHC é o estudo de como o humano opera a máquina; UX é o estudo de como o humano se sente ao usar o produto. O foco da IHC é o diálogo técnico e cognitivo, ou seja, relativo ao processo mental da percepção, memória, juízo, raciocínio. (cliques, tempo de resposta, legibilidade, modelos mentais). O foco da UX é a percepção total (valor percebido, confiança na marca, prazer e a jornada completa antes e depois do clique).

Boas e más experiências em UX

Veja alguns exemplos de situações que podem causar boas e más experiências para o usuário.

UX não é "deixar bonito". Design bonito não garante boa experiência.
Um sistema pode ser visualmente atraente e ainda assim ser confuso ou difícil de usar.

Má experiência
Usuário não entende o que fazer, comete erros ou sente frustração durante a tarefa.

Boa experiência
Usuário entende rapidamente o que fazer, conclui a tarefa com segurança e sente confiança no sistema.

Boa UX se conquista com boas decisões.
Cada texto, botão, fluxo ou mensagem é uma escolha que impacta o comportamento do usuário.

UX é evidência. Opinião é gosto. UX é método.
Decisões devem ser baseadas em pesquisa, testes e observação, não apenas em opinião. Experiência do usuário se constrói com dados e validação.

Princípios de Donald Norman

Os princípios de Norman são fundamentos cognitivos que sustentam o design de UX. Vejamos.

Nota. Originalmente, na **primeira edição** de seu livro, de **1988**, Don Norman fez a proposta de **seis** princípios. Já na **segunda edição** de seu livro, de **2013**, ele evidencia a existência de **sete princípios**. **Neste material, abordaremos os sete princípios**.

Nota. Conforme apresentamos exemplos de cada princípio, eles podem parecer se sobrepor, de modo que um item apresentado possa parecer exemplo de mais de um princípio. Isso, de fato, é verdade. Os princípios de Norman são como ingredientes de uma receita. Raramente aparecem sozinhos.

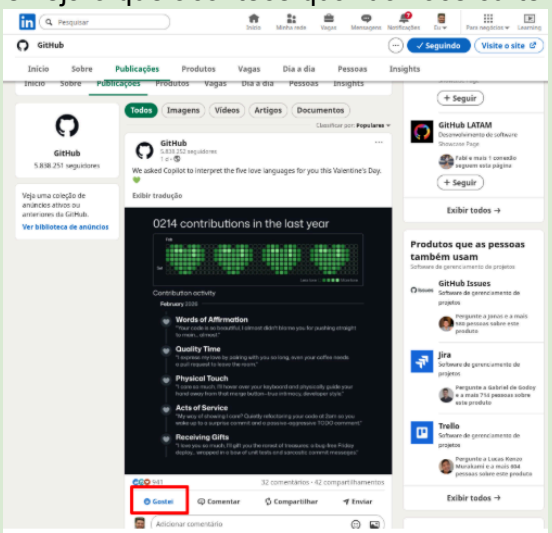
O princípio Feedback

Definição do princípio Feedback

O sistema precisa informar ao usuário que a ação foi realizada e qual foi o resultado. É a resposta imediata (um som, uma luz, uma vibração).

Um bom exemplo: https://www.linkedin.com/

Visite o site do LinkedIn e veja o que acontece quando você curte um post.



Um mau exemplo: Carrinho de compra de um e-commerce

Se imagine em um e-commerce qualquer. Você clica em um ícone e não tem feedback algum. Ainda que o site mostre um ícone de carrinho de compras (geralmente no canto superior direito), nada acontece. Nenhuma animação no carrinho. O ícone dele não muda. Não aparece um número ilustrando que a quantidade de itens aumentou.

O princípio Conceptual Model (Modelo Mental)

Definição do princípio Conceptual Model

O design deve ajudar o usuário a criar uma simulação mental de como o sistema funciona. Se o modelo mental do usuário for igual ao do designer, o uso será fácil. O exemplo mais clássico de Modelo Mental em texto é o Hiperlink (o famoso texto azul sublinhado). Por que isso é um modelo mental? Ao longo de décadas usando a Internet, você desenvolveu um modelo mental de que:

Texto com cor diferente (geralmente azul) + Sublinhado = Um portal para outra página.

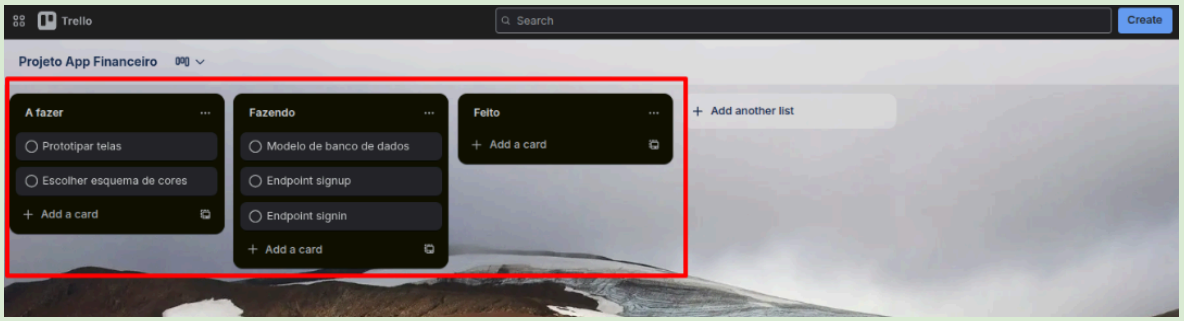
Ação esperada: Se eu clicar, serei levado a outro lugar.

Quando o modelo mental é respeitado:

Se você clica no texto azul e o navegador carrega uma nova página, o seu modelo mental foi confirmado. O sistema funcionou exatamente como você previu na sua cabeça.

Um bom exemplo: https://trello.com/

Visite o site do Trello. Ele mostra cartões dispostos em várias colunas. O que você deduz sem que seja necessário explicar, graças a eventuais experiências prévias gravadas na sua mente?



Um mau exemplo: O botão Iniciar do Windows

O botão Iniciar do Windows. Há muitas versões do Windows que requerem que o usuário clique em Iniciar para que possa desligar o computador.

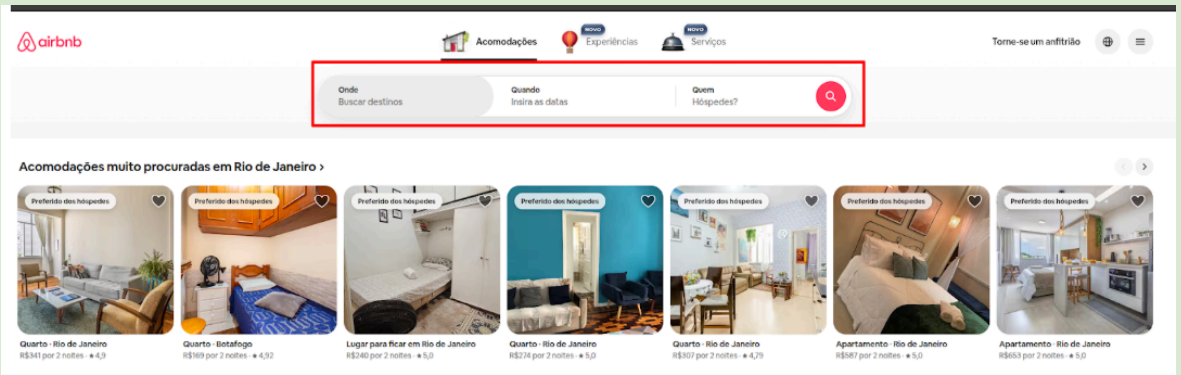
O princípio Affordance (Capacidade, disponibilidade, potencialidade)

Definição do princípio Affordance

Indica quais ações são possíveis em um objeto ou interface, ou seja, conjunto de propriedades físicas de um objeto ou ambiente que indicam, intuitivamente, como ele pode ser usado, agindo como pistas visuais para sua funcionalidade (ex: maçaneta convida a girar, botão convida a clicar).

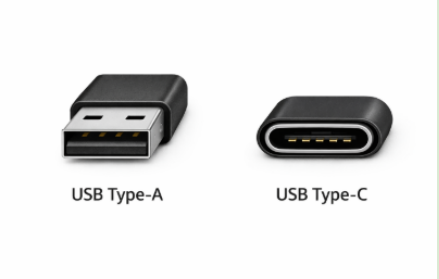
Um bom exemplo: https://www.airbnb.com.br/

Visite o site do Airbnb. Olhe para o campo de busca e seletores de data. Não parecem mero texto. Parecem clicáveis por serem delimitados por bordas e por trocarem de cor com a passagem do mouse.



Um mau exemplo: Entradas USB (Tipo A)

Um conector USB padrão Tipo A parece ser simétrico à primeira vista. Ele sugere a affordance de que pode ser inserido de qualquer lado. No entanto, ele só entra de um jeito. Isso é uma falha de design físico: a affordance sugere uma liberdade que a restrição física não permite. O conector USB padrão Tipo C corrigiu isso criando uma affordance que condiz com a realidade.



O princípio Signifiers (Significantes)

Definição do princípio Signifiers

São os sinais que indicam onde a ação pode/deve ocorrer. Enquanto a affordance é a capacidade, o significante é o sinalizador. Cursor de "Mãozinha": Quando você passa o mouse sobre um link e a seta vira uma mão com o dedo indicador. É um significante de que aquele elemento é clicável.

Um bom exemplo: https://www.instagram.com/

Visite o site do Instagram. Veja os ícones de coração, bolha de comentário e avião de papel abaixo de cada post. O coração não é a ação de curtir em si, mas é o sinal de que pode ser clicado para que ela aconteça.

Um mau exemplo: Uma caixa de texto que não exibe o cursor piscando

Quando estamos digitando, é natural que desejemos saber onde vai aparecer a próxima letra quando digitada. Por isso, é importante uma indicação visual de onde isso vai acontecer, o que geralmente é apresentado por meio de um cursor que fica piscando.

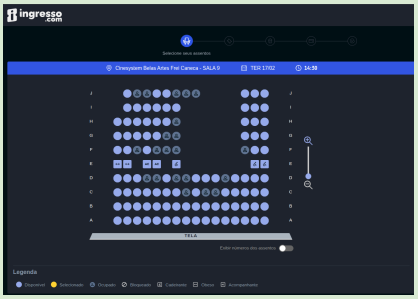
O princípio Mapping (Mapeamento)

Definição do princípio Mapping

Refere-se à relação entre os controles de um sistema e os seus efeitos no mundo ou na interface. O objetivo de um bom mapeamento é ser tão natural que o usuário não precise pensar ou memorizar como usar o controle. Ele utiliza analogias espaciais e padrões culturais para criar essa conexão. Temos um bom mapeamento quando a disposição física dos controles corresponde à disposição dos objetos controlados. Um exemplo clássico é a disposição dos botões do fogão combinando com a posição das bocas.

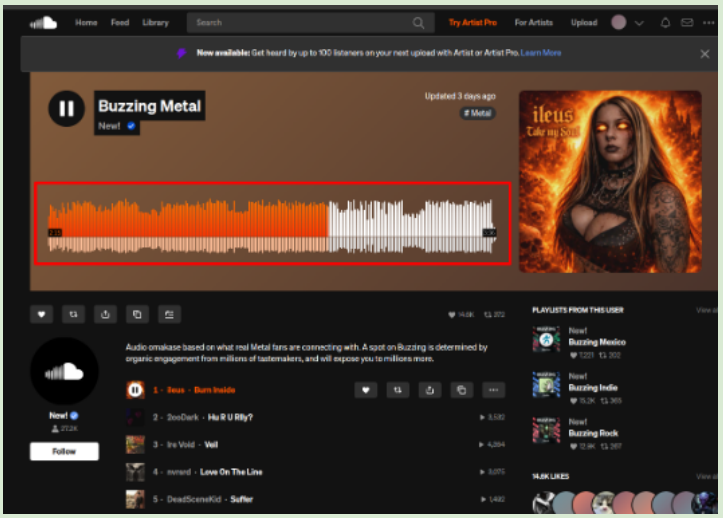
Um bom exemplo: https://www.ingresso.com.br/

Visite o site ingresso.com e clique em um filme qualquer, como se fosse comprar um ingresso. Observe como o site mostra a disposição dos assentos tal qual ela é fisicamente, inclusive destacando onde fica a tela.



Outro bom exemplo: https://soundcloud.com/

Visite o site do SoundCloud. Toque uma música. Veja que ele mostra as ondas (waveform) do áudio. O mapeamento aqui está relacionando a intensidade do som com o tempo. Se você vê uma montanha de ondas à frente, seu cérebro mapeia instantaneamente que uma parte barulhenta da música pode estar chegando.



Um mau exemplo: Chuveiro monocomando, presente em alguns hotéis

Um chuveiro assim possui uma única alavanca que podemos girar e puxar. Girar para a direita deixa mais quente ou fecha a água? Inclinar para cima aumenta a temperatura ou a pressão?

Outro mau exemplo: Scroll invertido

Este causou muita confusão quando, em 2011, a Apple introduziu o conhecido Natural Scrolling? O que acontece quando giramos a bolinha do mouse para cima ou para baixo? Claro, é configurável. Entretanto, ao introduzir esse novo modelo, foram quebrados anos de convenção e, portanto, a forma como o usuário pensava precisou ser alterada.

O princípio Constraints (Restrições)

Definição do princípio Constraints

São limites que impedem o usuário de cometer erros, restringindo as possibilidades de ação (ex: o conector micro USB que só entra de um lado).

Um bom exemplo: https://www.tesla.com/cybertruck/design#overview

Visite o site da Tesla e configure um veículo como se fosse comprá-lo. Conforme você interage, de acordo com suas escolhas, o site impõe restrições que impedem que você configure de alguma forma que não faça sentido.

Um mau exemplo:Campos de Formulário sem validações

Considere um campo de um formulário em que o usuário deva digitar uma data ou um valor numérico, como a sua idade. O campo deve estar associado a restrições que impeçam o usuário de digitar dados inválidos.

O princípio Discoverability (Facilidade de descoberta)

Definição do princípio Discoverability

É o grau de facilidade com que um usuário, ao interagir com um produto, consegue determinar quais ações são possíveis e como realizá-las. Se o usuário precisa "adivinhar" ou "tentar a sorte", a facilidade de descoberta é baixa.

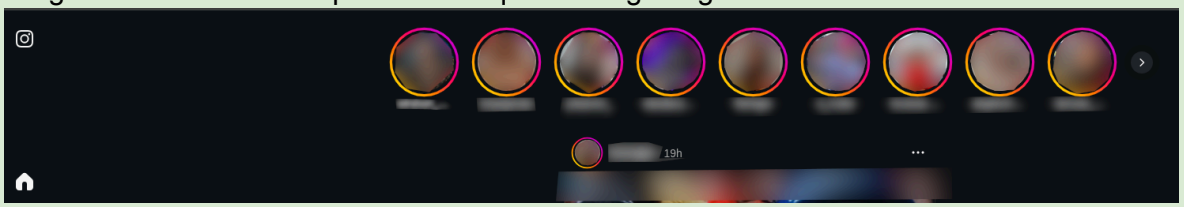
Um bom exemplo: https://www.instagram.com/

Quando você abre o Instagram e vê os círculos com as fotos dos seus amigos no topo, a facilidade de descoberta das funções é alta porque:

- O anel colorido ao redor da foto sinaliza que há algo novo (Significante).

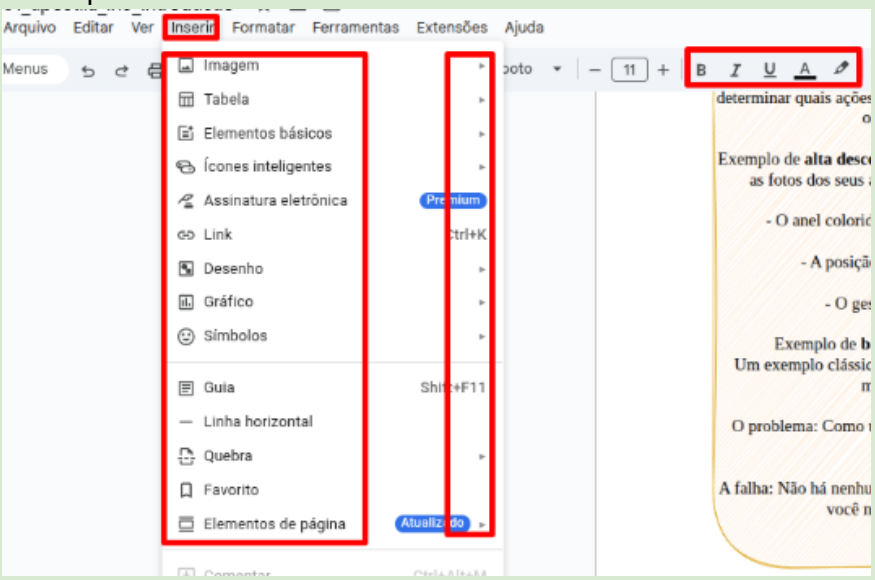
- A posição no topo da tela torna a função impossível de ignorar.

- O gesto de tocar é a resposta óbvia que o design sugere.



Outro bom exemplo (facilidade de descoberta hierárquica): https://docs.google.com/

Visite o site do Google Docs e verifique como as funcionalidades nos são entregues. As mais comuns estão explícitas. Outras, menos comuns, não estão explícitas, mas podem ser encontradas facilmente por meio de menus. É uma espécie de facilidade de descoberta hierárquica.



Um mau exemplo: uso de smartphones por meio de gestos

Como um novo usuário sabe que precisa "deslizar de baixo para cima e segurar no meio" para ver os apps abertos? Não há nenhum sinal visual (botão ou ícone) indicando que essa ação existe. Se você não souber previamente, a função está "escondida".

Nota. É comum **confundir facilidade de descoberta**, **signifiers** e, até mesmo, **affordance**. Para não confundir, veja dois exemplos.

Exemplo 1: Ponto de Ônibus

O signifier: É a placa com o desenho de um ônibus e a lista de horários.

A facilidade de descoberta: Graças à placa (signifier), uma pessoa que nunca esteve naquela rua consegue descobrir onde o ônibus para. Se tirarem a placa, a affordance de "lugar de parada" continua lá (o ônibus ainda pode parar), mas a facilidade de descoberta desaparece porque não há mais um signifier.

Exemplo 2: Barra de rolagem

O signifier: Aquela barrinha cinza fina que aparece no canto direito da tela.

A facilidade de descoberta: Ela permite que você descubra que existe mais conteúdo para baixo, mesmo que ele não esteja visível no momento.

Nota: Quando o Instagram ou o TikTok escondem a barra de rolagem, por exemplo, eles estão removendo o signifier e diminuindo a facilidade de descoberta em troca de uma interface mais "limpa".

Uma aplicação React que viola os princípios

Vamos desenvolver uma aplicação React que **viola todos os princípios de Don Norman**. Trata-se de uma aplicação que permite o **cadastro de músicas**.

O primeiro passo é criar a aplicação com

```
npm create vite@latest aula01-ihc-principios-don-norman -- --template react
```

É possível que apareça a sugestão de uso da versão experimental do Vite 8. Como estamos em um ambiente de aprendizado, façamos uso dela.

Também é possível que apareça a sugestão de já instalar as dependências com o npm e colocar o projeto em execução. Diga que deseja fazê-lo.

```
(base) rodrigo@insp-3530:~/workspaces/20261/preparo_aulas/react$ npm create vite@latest ihc-aula01-principios-don
orman -- --template react
Need to install the following packages:
create-vite@8.3.0
Ok to proceed? (y) y


> npx
> create-vite ihc-aula01-principios-don-norman --template react

│
◇  Use Vite 8 beta (Experimental)?:
│  Yes
│
◇  Install with npm and start now?
│  Yes
│
◇  Scaffolding project in /home/rodrigo/workspaces/20261/preparo_aulas/react/ihc-aula01-principios-don-norman...
│
◇  Installing dependencies with npm...
```

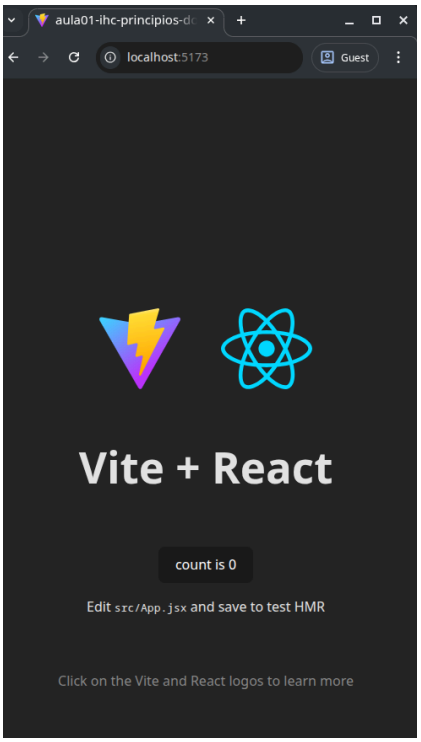
Caso não apareça a opção de instalar as dependências e executar o projeto, você pode fazê-lo com

cd aula01-ihc-principios-don-norman
npm i
npm run dev

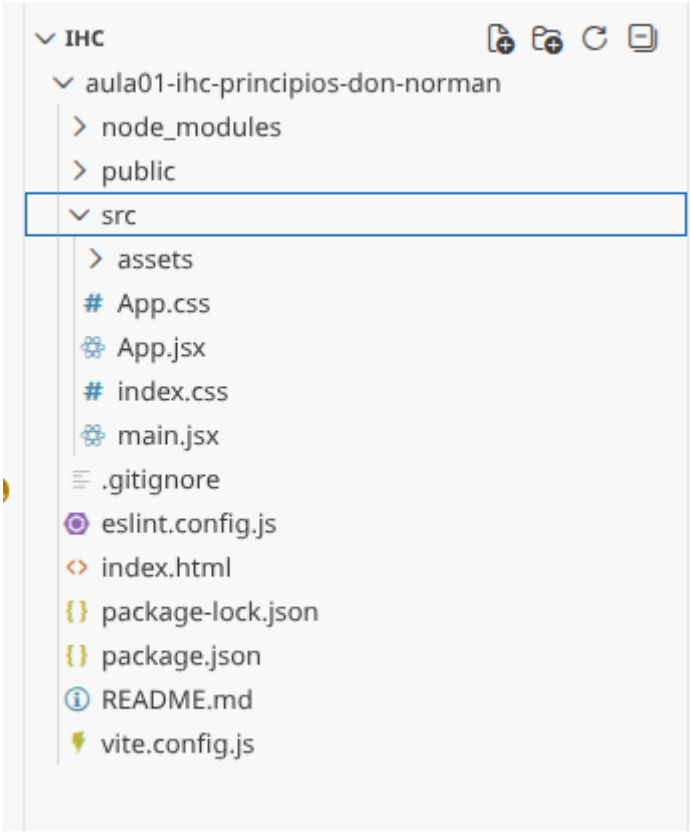
A seguir, basta abrir uma aba do navegador que preferir e visitar

localhost:5173

Veja.



A estrutura da aplicação é a seguinte.



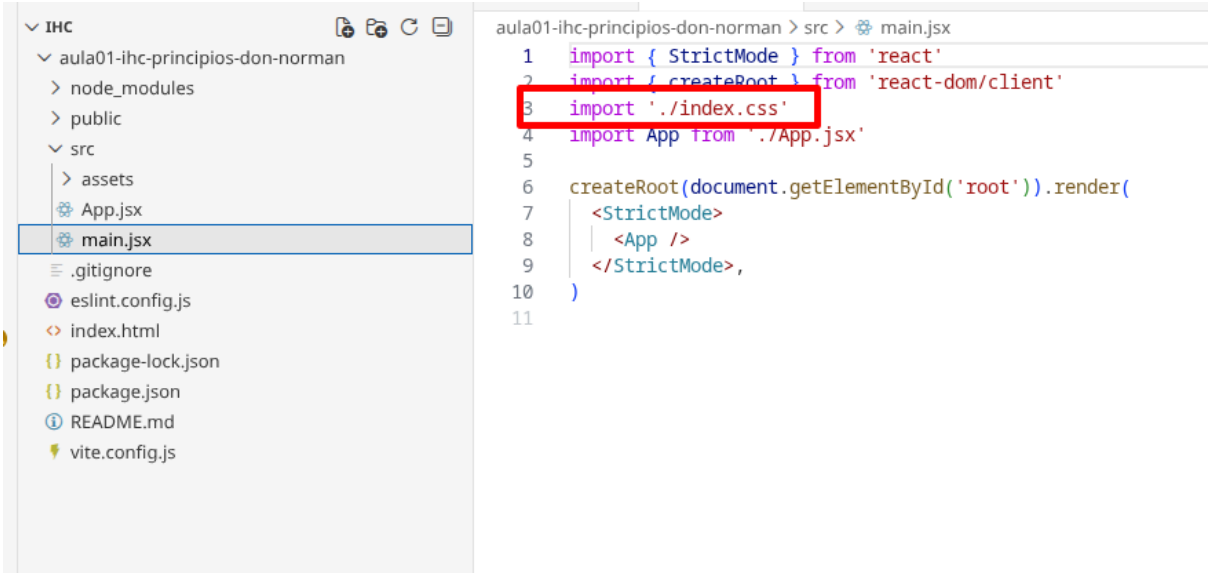
Faça os seguintes ajustes

- Apague o arquivo App.css
- Apague o arquivo index.css
- Apague o conteúdo do arquivo App.jsx
- Abra o arquivo App.jsx e adicione o seguinte conteúdo. Trata-se da definição de um componente React simples, apenas para garantir que a aplicação continua funcionando.

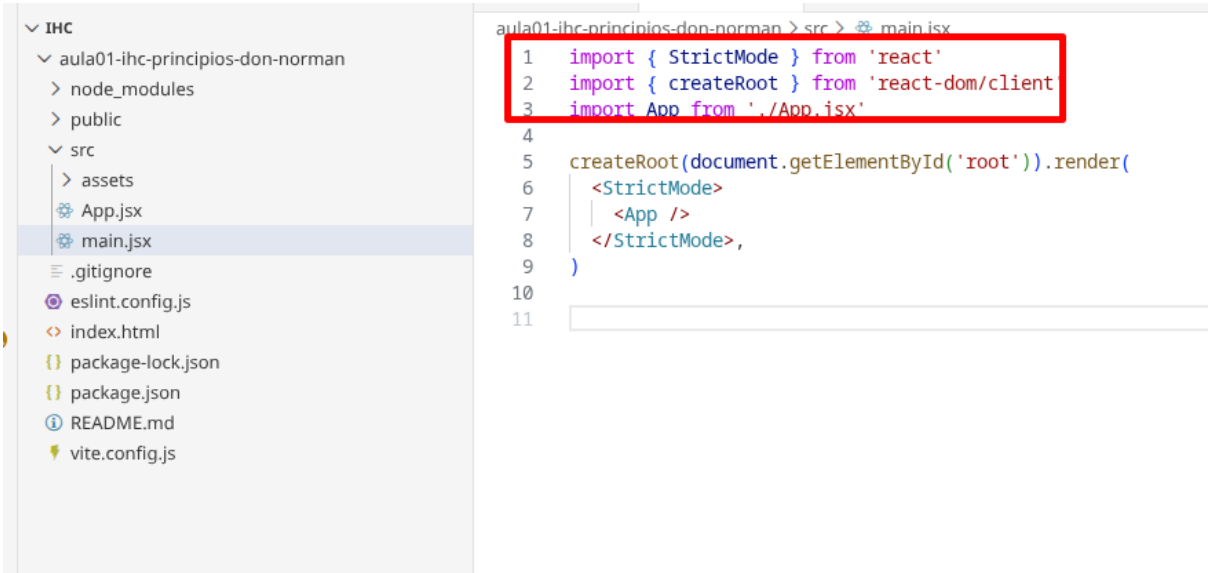
```
export default () => {
 return (
   <div>
     <h1>Olá, mundo!</h1>
   </div>
 );
}
```

- Abra o arquivo main.jsx e apague a linha que importa o arquivo index.css (a linha inteira). Afinal, ele não existe mais.

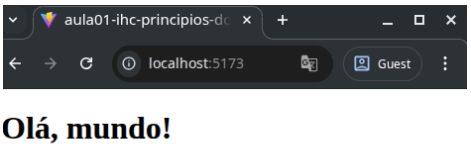
Originalmente ele é assim.



Fica assim.



Veja o resultado esperado no navegador.



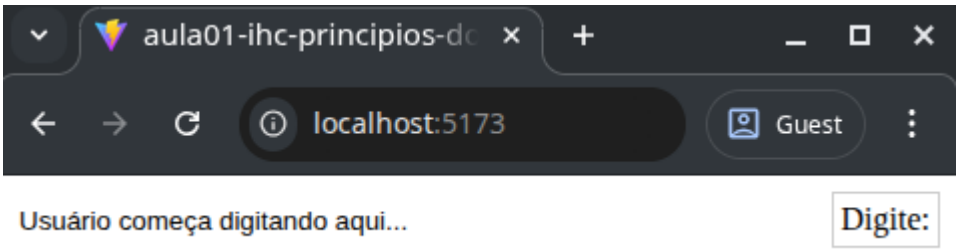
Clique com o direito na pasta src e crie um arquivo chamado CampoIncorreto.jsx. Veja seu conteúdo. Observe que os pontos em que as violações acontecem estão destacados como comentários.

```
export const CampoIncorreto = ({ valor, setValor, id }) => {
 return (
   <div style={{ display: 'flex', flexDirection: 'row-reverse',
marginBottom: '20px' }}>

     {/* Esse label tem duas violações */}
     {/* 1. VIOLAÇÃO: Mapeamento (Mapping) */}
     {/* 2. VIOLAÇÃO: Significantes (Signifiers) */}
     <label htmlFor={id} style={{ marginLeft: '8px', border: '1px solid
#ccc', padding: '4px'}}>Digite:</label>

     {/* 3. VIOLAÇÃO: Affordance */}
     <input
       id={id}
       type="text"
       value={valor}
       onChange={(e) => setValor(e.target.value)}
       style={{
         border: 'none',
         outline: 'none',
         color: '#000',
         width: '100%'
       }}
     />
   </div>
 );
};
```

Para testar, ajuste o conteúdo do arquivo App.jsx da seguinte forma. Ele passa a usar o componente CampoIncorreto momentaneamente.

```
import { CampoIncorreto } from "./CampoIncorreto";

export default () => {
 return (
   <div>
     <CampoIncorreto />
   </div>
 );
}
```

Veja o resultado esperado.



A seguir, vamos criar um componente React que representa um formulário. Crie um arquivo chamado FormularioIncorreto.jsx, também na pasta src. Veja seu conteúdo. Observe que ele faz uso do componente que criamos anteriormente.

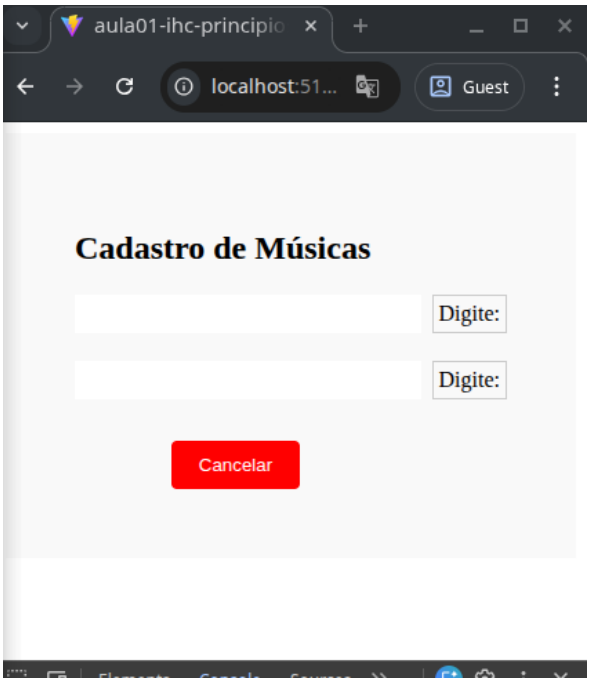
```
import { CampoIncorreto } from './CampoIncorreto';
import { useState } from 'react';
export const FormularioIncorreto = () => {
 const [musica, setMusica] = useState('');
 const [ano, setAno] = useState('');

 const handleEnviar = (e) => {
   e.preventDefault();

   // 4. VIOLAÇÃO: Feedback
   setMusica('');
   setAno('');
 };

 const handleCancelar = () => {

   // 5. VIOLAÇÃO: Modelo Mental (Conceptual Model)
   console.log("Salvando no banco de dados...", { musica, ano });
 };
```

```
 return (
   <form
     onSubmit={handleEnviar}
     style={{
       padding: '50px',
       backgroundColor: '#f9f9f9'
     }}>
     <h2>Cadastro de Músicas</h2>

     <CampoIncorreto id="campoMusica" valor={musica} setValor={setMusica} />

     {/* 6. VIOLAÇÃO: Restrições (Constraints) */}
     <CampoIncorreto id="campoAno" valor={ano} setValor={setAno} />

     <div style={{ display: 'flex', gap: '20px', marginTop: '30px' }}>

       {/* 7. VIOLAÇÃO: Facilidade de descoberta (Discoverability) */}
       <button
         type="submit"
         style={{
           backgroundColor: '#f9f9f9',
           color: '#f9f9f9',
           border: 'none',
           cursor: 'default'
         }}>
         Enviar
       </button>

       <button
         type="button"
         onClick={handleCancelar}
         style={{
           backgroundColor: 'red',
           color: 'white',
           padding: '10px 20px',
           border: 'none',
           borderRadius: '4px' }}>
         Cancelar
       </button>

     </div>
   </form>
 );
};
```

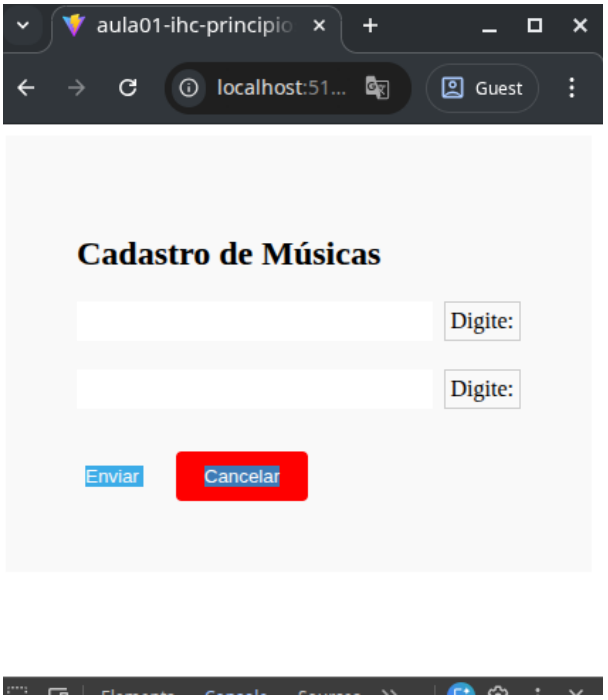
A seguir, atualizamos o conteúdo do arquivo App.jsx, utilizando o novo componente.

```
import { FormularioIncorreto } from "./FormularioIncorreto";
export default () => {
 return (
   <div>
     <FormularioIncorreto />
   </div>
 );
}
```

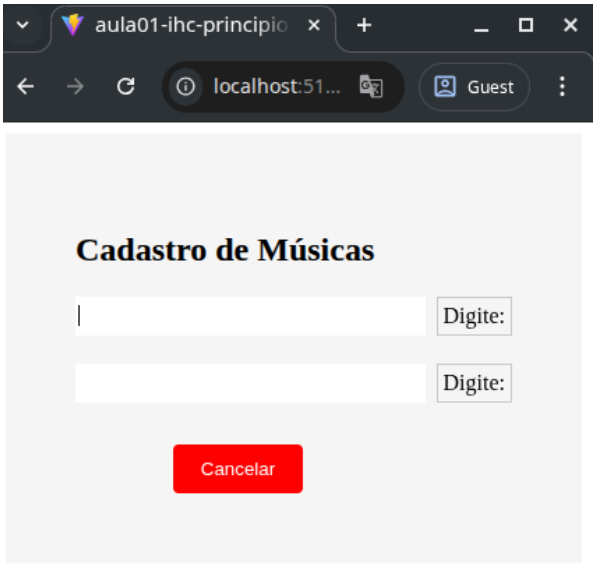
Veja o resultado esperado.

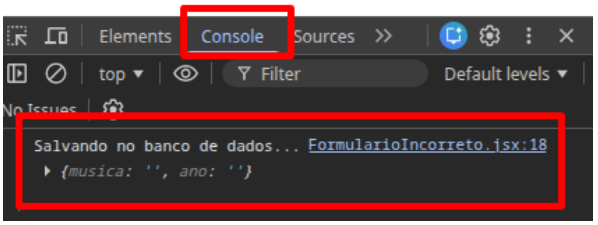


Observe que o botão de envio nem aparece. passe o mouse do lado esquerdo do botão de cancelar, tentando fazer uma seleção. Veja que ele pode aparecer assim.



Além disso, abra o Chrome Dev Tools com CTRL + SHIFT + I (ou equivalente do navegador que estiver utilizando) e verifique o que acontece quando o botão de cancelar é clicado.





Estamos diante de uma aplicação que viola os 7 princípios de Don Norman.

Referências

NIELSEN, Jakob. **Usability engineering**. Boston: Academic Press, 1993.

NORMAN, Don. **The design of everyday things: revised and expanded edition**. New York: Basic Books, 2013.

REACT. **React**. [S. l.]: Meta, [2026]. Disponível em: https://react.dev/. Acesso em: fev. 2026.